



EDUQA.PE

INTRODUCCIÓN A SQLITE CON PYTHON



SESIÓN 04

Transacciones, índices y rendimiento

AUTOR

Alejandro Valentino Seminario Medina

AI Engineer



Transacciones

Una transacción agrupa varias sentencias en una sola operación indivisible: o se aplican todas, o no se aplica ninguna.

Nota técnica. SQLite garantiza las propiedades ACID incluso ante un corte de energía o un fallo del sistema operativo. El módulo de Python abre y confirma las transacciones por su cuenta, con el comportamiento que define el parámetro `isolation_level` de `connect()`.

```
with conexion:
    conexion.execute("UPDATE cuentas SET saldo = saldo - ? WHERE id = ?", (100, 1))
    conexion.execute("UPDATE cuentas SET saldo = saldo + ? WHERE id = ?", (100, 2))
```



Transacciones

Si el bloque lanza una excepción, la transacción se revierte entera y ninguna de las dos actualizaciones queda aplicada.

```
try:
    with conexion:
        conexion.execute("DELETE FROM ventas")
        raise ValueError("algo salió mal")
except ValueError:
    print(conexion.execute("SELECT COUNT(*) FROM ventas").fetchone()[0])
```

Salida

12



Transacciones

Documentación: [TRANSACTION](#) · [Control de transacciones](#) · [La conexión como gestor de contexto](#)



Transacciones y velocidad

Cada sentencia suelta se ejecuta dentro de su propia transacción implícita, y cada transacción obliga a escribir en el disco. Agrupar muchas inserciones en una sola transacción reduce ese trabajo a una vez.

Nota técnica. La diferencia es de dos a tres órdenes de magnitud en cargas masivas. Es la primera causa a revisar cuando una carga de datos en SQLite parece inexplicablemente lenta.

Documentación: [TRANSACTION](#)



Índices

Un índice es una estructura auxiliar que permite localizar filas sin recorrer la tabla entera.

Nota técnica. SQLite usa árboles B para los índices. Aceleran las lecturas que filtran u ordenan por la columna indexada, y en contrapartida ocupan espacio y encarecen cada escritura, porque el índice se actualiza junto con la tabla.

```
conexion.execute("CREATE INDEX idx_ventas_vendedor ON ventas(vendedor_id)")
```

Documentación: **CREATE INDEX**



Comprobar si un índice se usa

La sentencia **EXPLAIN QUERY PLAN** muestra cómo el planificador piensa resolver una consulta, sin ejecutarla.

Nota técnica. En la salida, SCAN significa que recorre la tabla completa y SEARCH que usa un índice. Ver SCAN sobre una tabla grande en una consulta frecuente es la señal de que falta un índice.



Comprobar si un índice se usa

```
plan = conexion.execute("""
    EXPLAIN QUERY PLAN
    SELECT * FROM ventas WHERE vendedor_id = ?
""", (3,)).fetchall()

for paso in plan:
    print(paso["detail"])
```

Salida

```
SEARCH ventas USING INDEX idx_ventas_vendedor (vendedor_id=?)
```

Sin el índice, el mismo plan muestra un recorrido completo de la tabla.



Comprobar si un índice se usa

```
conexion.execute("DROP INDEX idx_ventas_vendedor")

plan = conexion.execute("""
    EXPLAIN QUERY PLAN
    SELECT * FROM ventas WHERE vendedor_id = ?
""", (3,)).fetchall()

print(plan[0]["detail"])
```

Salida

SCAN ventas

Documentación: [EXPLAIN QUERY PLAN](#)



Modo WAL

El modo de registro de escritura anticipada, **WAL**, cambia la forma en que SQLite guarda los cambios y permite que las lecturas no bloqueen a la escritura.

Nota técnica. En el modo por omisión, un escritor bloquea a todos los lectores. En WAL, los lectores siguen viendo la última versión confirmada mientras alguien escribe. Sigue admitiendo un solo escritor a la vez, y crea dos archivos auxiliares junto a la base.

```
print(conexion.execute("PRAGMA journal_mode = WAL").fetchone()[0])
```

Salida

wal



Modo WAL

Documentación: [Registro de escritura anticipada](#) · [Sentencias PRAGMA](#)



Fechas

SQLite no tiene un tipo de dato para fechas. Se guardan como texto en formato ISO 8601, y un conjunto de funciones del motor opera sobre ellas.

Nota técnica. El formato AAAA-MM-DD es el recomendado porque su orden alfabético coincide con el cronológico. Desde Python 3.12 los adaptadores automáticos para date y datetime están obsoletos: la documentación recomienda convertir a texto de forma explícita.



Fechas

```
from datetime import date

conexion.execute(
    "INSERT INTO ventas (fecha, producto) VALUES (?, ?)",
    (date(2026, 8, 23).isoformat(), "Laptop"),
)
```

```
print(conexion.execute("SELECT date('now', '+7 days')").fetchone()[0])
```

Salida

2026-08-12



Fechas

Documentación: [Funciones de fecha y hora](#) · [Adaptar tipos de Python](#)



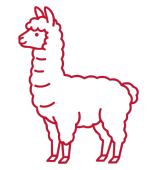
Un programa completo

Estas son las piezas anteriores en un solo archivo ejecutable.

```
import sqlite3

with sqlite3.connect(":memory:") as conexion:
    conexion.row_factory = sqlite3.Row
    conexion.execute("PRAGMA foreign_keys = ON")

    conexion.execute("""
        CREATE TABLE vendedores (
            id      INTEGER PRIMARY KEY,
            nombre TEXT    NOT NULL,
```



Un programa completo

```
        region TEXT      NOT NULL
    )
    """

conexion.executemany(
    "INSERT INTO vendedores (nombre, region) VALUES (?, ?)",
    [("Ana Quispe", "Norte"), ("Luis Mendoza", "Sur")],
)

for fila in conexion.execute("SELECT nombre, region FROM vendedores"):

    print(fila["nombre"], "-", fila["region"])
```



Un programa completo

Salida

Ana Quispe - Norte

Luis Mendoza - Sur

Documentación: [Guías de uso](#)



Cierre

Con esto ya es posible diseñar un esquema, consultarlo desde Python y hacerlo rendir. El paso siguiente natural es PostgreSQL: el lenguaje SQL es casi el mismo y la interfaz de Python también, porque ambos conectores siguen la misma especificación DB-API.

Los documentos sobre peculiaridades del motor y sobre cuándo usar SQLite merecen una lectura completa antes de llevar nada a producción.

Documentación: [Peculiaridades de SQLite](#) · [Usos apropiados](#) · [sqlite3](#)



Fin de la sesión

Transacciones, índices y rendimiento

Alejandro Valentino Seminario Medina